Семейства оптимизаций инференса

Карта для исследователя: что ускоряет каждое семейство, насколько оно перспективно у нас и с чего начать · Nemotron · октябрь 2026

Как читать. У каждого семейства есть паспорт: суть, оценка перспективности у нас, что оно ускоряет в теории, потери качества, где играть и сколько стоит. Дальше — как это работает, почему это важно именно для наших моделей, с чего начать, ловушки и что читать.
«В теории» — потолок по простой модели: при генерации по одному токену скорость упирается в память (раздел «Откуда берётся время ответа»). Реальный выигрыш обычно ниже. Любая цифра здесь — гипотеза, которую проверяет замер.
Шкала перспективности учитывает выигрыш на нашем железе, цену в GPU-часах и долларах, применимость к гибриду Mamba и MoE, риск и то, чему научимся: ●●●●● брать первым · ●●●●○ сильный кандидат · ●●●○○ после основы, есть шанс на свой результат · ●●○○○ только как часть другого или если останется время · ●○○○○ сейчас не тратить время.

Сводная таблица

Щёлкните по названию, чтобы перейти к семейству. Потолки — по модели из раздела «Откуда берётся время ответа», для 9B и 30B-A3B в Q4, если не сказано иначе.

Семейство	Что ускоряет	Потолок в теории	Потери	Перспективность
1. Спекулятивный декодинг	проходов цели на токен	1,3–1,5× с черновиком 4B; 2–4× с дешёвым черновиком при приёме 0,7–0,9	нет	●●●●● 5
2. Экономия рассуждений	число токенов N	1,2–5× (DEER: −19…80% длины)	есть	●●●●● 5
3. Квантизация весов	байты на токен	F16→Q8 1,9×; Q8→Q4 1,75×	малые	●●●●○ 4
4. Приёмы для MoE	байты на токен	эксперты 6→4: 1,15×; 6→3: 1,25×	есть	●●●●○ 4
5. Nano→Super: каскады и SpecReason	токены и деньги Super	SpecReason 1,4–3× (статья); каскад в 1,3–2 раза дешевле	есть	●●●●○ 4
6. Разреженность ReLU^2	байты на токен	1,1–1,25× точно; до 1,6× с предсказателем	нет / есть	●●●○○ 3
7. Движок, сборка, флаги	накладные расходы	до потолка из таблицы потолков	нет	**шаг 0**
8. Пропуск и удаление слоёв	байты на токен	−25% слоёв → 1,33×	большие	●●○○○ 2
9. Параллельная генерация	ответов за проход	k ответов почти по цене одного	нет	●●○○○ 2
10. Сжатие KV-кэша и состояния	трафик KV	1,02× при 8 тыс. токенов; 1,3× при 128 тыс.	малые	●○○○○ 1
11. Сжатие промпта, кэш префикса	prefill	prefill — 0,6–4% времени на наших задачах	есть	●○○○○ 1
12. Сжатие с переобучением	всё сразу	большой, но 10^{10}–10^{11} токенов обучения	есть	●○○○○ 1

Семейство 7 — не тема исследования, а обязательная база: без неё любое «ускорение» может оказаться ускорением слабого движка.

Откуда берётся время ответа

Время ответа ≈ **prefill** (чтение промпта) + **N × t**, где N — число сгенерированных токенов, а t — время на один токен. При генерации для одного пользователя (батч 1) t ≈ байты, прочитанные из памяти за токен ÷ пропускная способность памяти (ПСП) + накладные расходы. Вычислитель успевает больше, чем память успевает подать: у T4 на каждый байт памяти приходится ~203 операции, а одному потоку в Q4 нужно всего ~3,3. Поэтому генерация «упирается в память», и почти все методы ниже либо читают меньше байт, либо делают меньше дорогих проходов, либо генерируют меньше токенов.

Рычаг	Что меняет	Семейства
Меньше байт на токен	t	3 квантизация · 4 MoE · 6 разреженность · 8 пропуск слоёв
Больше токенов за дорогой проход	t на токен ответа	1 спекулятивный декодинг
Меньше токенов	N	2 экономия рассуждений · 5 каскады Nano→Super
Меньше накладных расходов	t	7 движок и флаги
Быстрее prefill	prefill	11 сжатие промпта и кэш префикса
Больше ответов за проход	пропускная способность	9 параллельная генерация

Потолок скорости, токенов в секунду (ПСП ÷ байты на токен)

Модель и квант	ГБ на токен	T4 320 ГБ/с	780M DDR5-5600	780M LPDDR5X	Mac M1 68 ГБ/с	Mac M2–M3 100 ГБ/с	ПК DDR4 51 ГБ/с
4B · Q4_K_M	2,14	149	42	56	32	47	24
4B · Q8_0	3,76	85	24	32	18	27	14
9B · Q4_K_M	5,03	64	18	24	14	20	10
9B · Q8_0	8,82	36	10	14	8	11	6
9B · F16*	16,60	19	5	7	4	6	3
30B-A3B · Q4_K_M**	2,12	151	42	57	32	47	24
30B-A3B · Q8_0	3,72	86	24	32	18	27	14

*Байты на токен посчитаны по архитектуре: все веса, кроме входного эмбеддинга; у 30B-A3B — только активные ~3,5 млрд параметров. * 9B в F16 (16,6 ГБ) не помещается в одну T4; на двух картах, разбитых по слоям, потолок не растёт — половины читаются по очереди. ** 30B-A3B в Q4 на Kaggle помещается только в две T4. На практике получается обычно 50–80% от потолка.*

> **Привычка для каждого замера:** считать долю от потолка = ток/с × байты на токен ÷ ПСП. Если вышло 30%, есть что выжимать движком и флагами (семейство 7). Если 80% — помогут только методы, которые уменьшают байты, число проходов или число токенов.
> **Архитектурное наблюдение.** 30B-A3B в Q4 читает на токен ~2,1 ГБ — как плотная 4B и в 2,4 раза меньше, чем 9B. На ноутбуке её потолок ~42 ток/с против ~18 у 9B. Если замер это подтвердит, MoE-модель окажется и сильнее, и быстрее.

Что в наших моделях меняет приоритеты

Это важнее любой статьи: большинство методов придумано для чистых трансформеров с SwiGLU, а у нас гибрид Mamba-2 с вниманием, ReLU^2 и MoE.

Особенность	Цифры	Что из этого следует
Внимания почти нет	9B: 56 слоёв — 27 Mamba-2, 4 внимания, 25 MLP. 4B: 42 слоя — 21, 4, 17. 30B-A3B: 52 слоя — 23 Mamba-2, 23 MoE, 6 внимания	KV-кэш 9B — 16 КБ на токен: 134 МБ на 8 тыс. токенов против 5 ГБ весов в Q4. Методы для KV-кэша почти не нужны, а отсутствие FlashAttention на T4 почти не мешает

Особенность	Цифры	Что из этого следует
Состояние Mamba	9B: ~142 МБ в fp32, не растёт с длиной; читается и пишется на каждом токене (~6% трафика Q4-весов)	При отказе черновика состояние откатывают: копия на T4 ≈ 0,4 мс. Деревья черновиков требуют особых ядер — берём цепочки
ReLU^2 в MLP и экспертах	MLP без гейта: up → ReLU^2 → down	Точные нули после активации — естественная разреженность (семейство 6)
Модели рассуждающие	ответы на тысячи токенов; встроенный бюджет на рассуждение	N часто важнее t (семейство 2). Рекомендуемая выборка: 9B с рассуждением — T = 0,6, top-p = 0,95, без рассуждения — жадно; 30B-A3B с рассуждением — T = 1,0
MoE: 6 из 128 экспертов + 1 общий	маршрутизируемые эксперты — ~39% читаемого на токен	Можно брать меньше экспертов; проверка нескольких токенов задействует больше разных экспертов — спекуляция выигрывает меньше (семейство 4)
Словарь 131 тыс. токенов	выходной слой — 7% читаемых весов у 9B и 12% у 4B	Отдельная ручка для квантизации и для черновиков
Железо	T4 — sm75, 16 ГБ, 320 ГБ/с, две карты без NVLink. 780M — Vulkan. Mac — Metal. ПК — только CPU	На T4 нет bf16 (только fp16 — проверить на переполнения), FP8, FlashAttention-2 и ядер Marlin. Ноутбук и Mac — это llama.cpp

1. Спекулятивный декодинг

Суть	Дешёвый черновик предлагает γ токенов, большая модель (цель) проверяет их за один проход и оставляет совпавшие. Ответ тот же, что и без черновика.
Перспективность	●●●●● **5 из 5** — без потерь, работает на всех устройствах, в llama.cpp уже есть почти всё; для Super — главный способ «ускорить через Nano»
Что ускоряет в теории	Число дорогих проходов цели. За проход в среднем $E = (1 - \alpha^{\gamma+1}) \div (1 - \alpha)$ токенов, ускорение $S = E \div (\gamma \cdot c + v)$. Здесь α — доля принятых токенов, c — цена шага черновика относительно шага цели, v — цена проверки (≈1,05 у плотной модели).
Потери качества	Нет. При жадной проверке ответ совпадает с базой побайтно, при сэмплинге работает правило приёма min(1, p/q).
Где играть	Везде. Super — только офлайн: по logprobs API считаем, сколько токенов черновика он бы принял.
Цена	Замер одной конфигурации — ~0,5–1 GPU-ч на Kaggle или бесплатно на ноутбуке. Офлайн-приём для Super — ~$21 на всю программу, с запасом ~$32.

Потолки ускорения при разной доле принятых токенов

Черновик	c	α = 0,6	α = 0,7	α = 0,8	α = 0,9
Голова EAGLE или MTP	0,05	1,84× (γ=4)	2,27× (γ=6)	2,99× (γ=8)	4,22× (γ=8)
4B → 9B, оба в Q4	0,43	1,08× (γ=1)	1,15× (γ=1)	1,28× (γ=2)	1,49× (γ=4)
4B в Q3_K_M → 9B в Q4	0,34	1,15× (γ=1)	1,26× (γ=2)	1,42× (γ=3)	1,70× (γ=5)
Само-черновик, 50% слоёв	0,50	1,03× (γ=1)	1,10× (γ=1)	1,19× (γ=2)	1,35× (γ=3)
4B → 9B на двух T4, асинхронно (идеал)	0,43	1,87× (γ=2)	2,09× (γ=2)	2,32× (γ=2)	2,69× (γ=3)

Лучшая γ подобрана для каждой клетки, v = 1,05. Асинхронный вариант — идеал, когда черновик на второй T4 работает, пока цель проверяет; в статье AMUSD реальный прирост к обычной спекуляции — +29%. N-граммы почти ничего не стоят (c ≈ 0), но черновик есть не всегда: на повторяющемся тексте (правки кода, RAG, пересказ) выигрыш бывает 2× и больше, в свободном чате — около 1×.

Варианты

Вариант	Где	Оценка	Комментарий
N-граммы: ngram-mod, ngram-cache, ngram-simple; SuffixDecoding, PLD	везде	●●●●● 5	Бесплатно и без второй модели. Сила зависит от повторяемости текста — ищем, на каких задачах выигрыш.
Черновик-модель 4B → 9B	Kaggle, ноутбук	●●●●○ 4	4B дистиллирована из 9B — приём должен быть высоким, но черновик тяжёлый. Потолок ~1,3–1,5×; черновик в Q3_K_M дешевле — см. таблицу.
Nano 9B, Nano 30B, Lightning → Super, офлайн	API	●●●●● 5	Группа 3. Lightning учили на логитах Super — кандидат №1. Проверить, совпадают ли токенизаторы.
MTP у Lightning (draft-mtp)	Kaggle, ноутбук	●●●●○ 4	Встроенный черновик почти без цены. На MoE выигрыш ограничен ценой проверки (семейство 4).
Само-черновик пропуском слоёв: SWIFT, Draft & Verify	прототип в HF	●●●○○ 3	Одна модель в памяти, без обучения. На гибридах с Mamba, похоже, не проверяли — шанс на свой результат.
Асинхронно на двух T4: AMUSD, PEARL	Kaggle	●●●○○ 3	Подогнано под 2× T4. В llama.cpp нет — нужен свой цикл, а у гибрида откат нужен и черновику.
Lookahead (итерации Якоби)	прототип	●●○○○ 2	Без черновика, но проверяет несколько веток сразу — для состояния Mamba это сложно.
Обученные головы: EAGLE-3, DFlash, Medusa	Kaggle (обучение)	●●○○○ 2	DFlash даёт до 6× на трансформерах, но готовых голов для гибридных Nemotron мы не нашли, а учат их на H200. Мини-версия — второй этап (семейство 12).

Почему это важно у нас

- llama.cpp (сборка от 15.10.2026) уже умеет: `--spec-type` draft-simple, draft-mtp, draft-eagle3, draft-dflash, ngram-*; откатывает состояние для nemotron_h и nemotron_h_moe; код EAGLE-3 учитывает гибридные цели. Основная работа — мерить и объяснять, где и почему есть выигрыш.
- 4B получена из 9B обрезкой и дистилляцией (63 + 150 млрд токенов) с замороженной 9B в роли учителя. Её распределение близко к 9B, но шаг стоит ~0,43 шага цели — отсюда потолок.
- Состояние Mamba при отказе откатывают; копия ~142 МБ на T4 — около 0,4 мс на шаг проверки.
- Сервер llama.cpp возвращает в ответе timings: скорость, предложенные и принятые токены черновика (draft_n, draft_n_accepted) — этого хватает для кривых S(γ).

С чего начать

- База и n-граммы на шести задачах Spec-Bench (диалог, перевод, пересказ, вопросы, математика, RAG): где выигрыш, а где нет.
- Черновик 4B при γ = 1…6 — кривая S(γ); на Kaggle черновик на второй карте. Отдельно — 4B в Q3_K_M.
- Lightning с draft-mtp при γ = 1…4. Нужен GGUF с сохранёнными MTP-слоями: llama.cpp умеет скачивать их отдельным файлом.
- Офлайн для Super: доля совпадений жадных токенов Nano 9B, Nano 30B и Lightning с ответами Super.

```
# база и n-граммы: одинаковые промпты, одна сессия, одно устройство
llama-server -m nano9b-Q4_K_M.gguf -ngl 99 -fa on
llama-server -m nano9b-Q4_K_M.gguf -ngl 99 -fa on --spec-type ngram-mod
# черновик 4B; на Kaggle — на второй T4
llama-server -m nano9b-Q4_K_M.gguf -md nano4b-Q4_K_M.gguf -ngl 99 -ngld 99 \
    --spec-draft-n-max 4 --spec-draft-device CUDA1
# MTP у Lightning
llama-server -m lightning-Q4_K_M.gguf -ngl 99 --spec-type draft-mtp --spec-draft-n-max 2
```

Ловушки

- По умолчанию `--spec-draft-n-max` = 3 — γ надо подбирать: кривая S(γ) — главный график семейства.
- Приём в жадном режиме и при T = 0,6 разный — режим фиксируется в конфиге эксперимента.
- Черновик и цель должны токенизировать одинаково. Словарь у всех 131 072 токена, но этого мало — сравнить токенизацию на тех же текстах.
- Сравнивать в одном движке, на одном устройстве, в одной сессии; первый прогон — прогрев.

Что читать

- Leviathan et al., 2022 — основа и формула выше.
- Xia et al., 2024 — обзор семейства и Spec-Bench.
- STree — деревья черновиков для гибридов; The Mamba in the Llama — спекуляция для Mamba.
- Lookahead, Medusa, EAGLE-3, DFlash — для общего кругозора.

2. Экономия рассуждений

Суть	Та же модель, но меньше токенов рассуждения до ответа.
Перспективность	●●●●● **5 из 5** — самый большой рычаг для времени до ответа; без обучения; работает на любом железе и через API — для Super это единственное ускорение, которое мерится точно
Что ускоряет в теории	Число токенов N. Ускорение = $N_{было} \div N_{стало}$: −30% токенов → 1,4×, −50% → 2×. DEER сокращал длину на 19–80% — это 1,2–5×.
Потери качества	Есть — меряем точность. Результат — кривая «точность против числа токенов», а не одно число.
Где играть	Везде; для 30B-A3B, Lightning и Super — через API.
Цена	Через API — копейки: 200 задач × 3 тыс. токенов = 0,6 млн выходных токенов, это ~$0,14 у Nano 30B и ~$0,54 у Super. Локально — прогоны качества батчем (семейство 9).

Модель рассуждает внутри <think> … </think>, потом отвечает. У рассуждающих моделей рассуждение часто в десятки раз длиннее ответа, поэтому время до ответа — это в основном N. Бюджет в Nemotron устроен так: после N токенов модель дописывает текущую строку и ставит </think>; если перевода строки нет, тег вставляется принудительно через N + 500 токенов.

Варианты

Вариант	Как включить	Оценка	Комментарий
Встроенный бюджет	`--reasoning-budget N` в llama.cpp; reasoning_budget у 30B-A3B	●●●●● 5	9B учили на обрезанных рассуждениях (~5% данных): при коротком бюджете формат ответа не ломается.
Без рассуждения	/no_think у 9B; enable_thinking=False у 30B-A3B	●●●●● 5	Нижняя граница: самый быстрый вариант и обычно самый неточный.
Ранний выход: DEER	в точках вроде «Wait» пробный ответ; если уверенность высокая — стоп	●●●●○ 4	Без обучения; нужны logprobs — локально или через API. В статье −19…80% длины без потери точности на 11 моделях.
NoThinking + k ответов	без рассуждения, k ответов параллельно, выбор лучшего	●●●●○ 4	На нашем железе k ответов стоят почти как один (семейство 9).
Budget forcing (s1)	продлевать рассуждение словом «Wait»	●●○○○ 2	Ручка в обратную сторону — для точности, не для скорости. Полезна, чтобы понять форму кривой.

С чего начать

- 9B, 100–200 задач (GSM8K, MATH-500, часть AIME и GPQA): бюджет 0 / 256 / 512 / 1024 / 2048 / без ограничений → кривая «точность — токены».
- То же для 30B-A3B и Super через API; деньги считаем заранее.
- DEER и NoThinking + 4 ответа на тех же задачах — сравнить со встроенным бюджетом.

```
llama-server -m nano9b-Q4_K_M.gguf -ngl 99 -fa on --reasoning-budget 512
```

Ловушки

- Мерить время до ответа и точность, а не ток/с.
- На 100 задачах точность скачет на несколько процентов — нужны интервалы и одни и те же задачи для базы и варианта.
- У API max_tokens по умолчанию 8192: длинное рассуждение обрежется — это ошибка, а не экономия.
- Рекомендованные температуры разные (9B — 0,6, 30B-A3B — 1,0) — фиксировать в конфиге.

Что читать

- Stop Overthinking — обзор семейства.
- DEER, NoThinking, s1 — ссылки в таблице выше.
- Nemotron Nano 2 — как устроен и как обучен бюджет.

3. Квантизация весов

Суть	Хранить веса в 8, 6, 5, 4 и меньше битах вместо 16.
Перспективность	●●●●○ **4 из 5** — выигрыш надёжный и крупный, но тема хорошо изучена; своё — в смешанной разрядности для гибрида. Как база обязательна: без неё 9B не помещается в одну T4, а 30B-A3B — в ноутбук
Что ускоряет в теории	Байты на токен. Потолок — отношение бит на вес: F16 → Q8_0 — 1,88×; Q8_0 → Q6_K — 1,3×; Q8_0 → Q4_K_M — 1,75×; Q4_K_M → Q3_K_M — 1,24×.
Потери качества	Малые до ~4–5 бит на вес; ниже 4 — заметные, особенно в рассуждениях.
Где играть	Везде. Квантовать можно на CPU — на ноутбуке или в CPU-сессии Kaggle.
Цена	Готовые GGUF бесплатны; своя квантизация — CPU-часы; качество — ~1 GPU-ч на вариант.

Веса делят на блоки (в GGUF — по 32 или 256 значений) и хранят целые числа плюс масштаб на блок. k-кванты llama.cpp смешивают разрядность по тензорам, а матрица важности (imatrix) подбирает квантование по активациям на примерах. В HF есть bitsandbytes (NF4, INT8), GPTQ и AWQ.

Варианты

Вариант	Где	Оценка	Комментарий
Готовые GGUF: Q8_0 … Q3_K_M	везде	●●●●● 5	База всех замеров. Записывать автора, версию llama.cpp и хэш файла.
Смешанная разрядность по типам слоёв	везде	●●●●○ 4	Своё для гибрида: Mamba, внимание, MLP, эксперты, выходной слой — кто сколько бит переносит.
Калибровка imatrix	ноутбук, Kaggle	●●●●○ 4	Калибровать на рассуждениях или на обычном тексте — что лучше для наших задач.
2–3 бита для черновика	везде	●●●○○ 3	Качество черновика важно меньше: дешевле шаг с, а ответ не меняется (семейство 1).
bitsandbytes, GPTQ, AWQ в HF	Kaggle	●●○○○ 2	Для прототипов в HF; по скорости обычно уступают llama.cpp.

Почему это важно у нас

- Части гибрида по-разному чувствительны. NVIDIA при сжатии 12B → 9B заметила, что обрезка голов Mamba сильно бьёт по точности, — возможно, проекции Mamba хуже переносят и мало бит. Это гипотеза для замера.
- Для Mamba есть Quamba2 (W4A8 и W4A16, генерация до ~3× быстрее на A5000). Её ядра — в стиле Marlin, на T4, скорее всего, не пойдут, но выводы о чувствительности слоёв переносятся.
- NVIDIA советует держать состояние Mamba в fp32 (в vLLM — `--mamba_ssm_cache_dtype float32`). В llama.cpp оно всегда fp32; цену fp16 можно померить в HF или vLLM.
- На T4 при батче 1 важна память, а не арифметика: выгоднее квантовать только веса (W4A16), чем веса и активации (W8A8).

С чего начать

- 9B и 4B в Q8_0, Q6_K, Q5_K_M, Q4_K_M, IQ4_XS, Q3_K_M: llama-bench (скорость), llama-perplexity и 100 задач (качество) → таблица «бит — скорость — качество».
- Смешанная разрядность: в llama-quantize имя тензора в `--tensor-type` — регулярное выражение. Например, ssm_in и ssm_out в Q6_K, остальное в Q4_K_M; выходной слой — `--output-tensor-type`.
- imatrix на 200 рассуждениях против imatrix на обычном тексте.

```
llama-imatrix -m nano9b-F16.gguf -f reasoning_calib.txt -o imatrix.dat
llama-quantize --imatrix imatrix.dat --tensor-type "ssm_(in|out)=q6_k" \
    --output-tensor-type q6_k nano9b-F16.gguf nano9b-mix.gguf Q4_K_M
llama-bench -m nano9b-mix.gguf -ngl 99 -fa 1 -p 512 -n 128
```

Ловушки

- Перплексия мало говорит о рассуждениях — мерить и на задачах.
- 9B в F16 для калибровки весит ~18 ГБ: на ноутбуке помещается, на одной T4 — нет.
- Сравнивать кванты одного автора и одной версии llama.cpp.

4. Приёмы для MoE (группа 2)

Суть	В MoE-слое на токен работают 6 из 128 экспертов и общий эксперт. Можно брать меньше экспертов, квантовать эксперты отдельно, по-разному раскладывать их в памяти и подбирать спекуляцию под MoE.
Перспективность	●●●●○ **4 из 5** — главная тема группы 2: часть вариантов пробуется одним флагом, а связка MoE и спекуляции мало изучена
Что ускоряет в теории	Байты на токен. Маршрутизируемые эксперты — ~39% читаемого на токен у 30B-A3B: 6 → 5 экспертов — 1,07×; 6 → 4 — 1,15×; 6 → 3 — 1,25×.
Потери качества	Есть при сокращении экспертов и при грубом квантовании экспертов; нет при размещении в памяти.
Где играть	Kaggle (30B-A3B в Q4 — на двух T4) и ноутбук (32 ГБ общей памяти).
Цена	Как у семейства 3: ~0,5–1 GPU-ч на вариант.

Спекуляция и MoE

Проверка m токенов задействует объединение их экспертов. Если маршрутизация независима, проверка 4 токенов читает ~22 эксперта на слой вместо 6 и стоит ~2,1 шага, а не ~1. Тогда даже дешёвый черновик (c = 0,05) даёт 1,2–1,6× вместо 2,3–4,2× у плотной модели. Это оценка сверху по цене проверки: соседние токены часто выбирают одних и тех же экспертов. На A100 у похожей Qwen3-Coder-30B-A3B с EAGLE-3 измерено до ~2,3× — часть времени там уходит на расходы, не зависящие от числа экспертов. Вывод: короткие черновики (γ = 1–3) и замер.

Токенов в проверке	1	2	3	4	5	8
Разных экспертов на слой	6,0	11,7	17,2	22,4	27,3	40,8
Цена проверки, в шагах	1,00	1,37	1,73	2,07	2,40	3,28

Модель только памяти, независимая маршрутизация, 6 из 128 экспертов.

Варианты

Вариант	Где	Оценка	Комментарий
Меньше активных экспертов, без обучения	Kaggle, ноутбук	●●●●○ 4	Один флаг. Меняет ответы — мерить точность.
Спекуляция под MoE	Kaggle, ноутбук	●●●●○ 4	Короткие черновики, MTP у Lightning, n-граммы. Свести с семейством 1.
Эксперты в меньшей разрядности	везде	●●●○○ 3	Эксперты — ~93% всех весов 30B-A3B, но на токен дают ~39% читаемых байт: сильно экономит память, умеренно — время.
Число экспертов по уверенности роутера	прототип	●●●○○ 3	Not All Experts are Equal: пропускать слабых экспертов. Нужен свой код.
Эксперты в RAM (--n-cpu-moe)	Kaggle	●○○○○ 1	Нужно, только если модель не помещается в GPU; Mixtral-offloading — ~2 ток/с даже на T4.

С чего начать

- Число активных экспертов 6 / 5 / 4 / 3 через `--override-kv` → скорость и точность.
- Lightning: draft-mtp и n-граммы при γ = 1…4; сравнить выигрыш с плотной 9B.
- Эксперты в Q3_K, остальное в Q4_K_M — сколько теряем и выигрываем.
- Сравнить 30B-A3B и 9B на ноутбуке по скорости и точности — проверка архитектурного наблюдения из раздела «Откуда берётся время ответа».

```
# число активных экспертов без переобучения (llama-bench этот флаг не принимает)
llama-server -m nano30b-Q4_K_M.gguf -ngl 99 --override-kv nemotron_h_moe.expert_used_count=int:4
# эксперты отдельно в 3 битах
llama-quantize --imatrix imatrix.dat --tensor-type "ffn_(up|down)_exps=q3_k" \
    nano30b-F16.gguf nano30b-mix.gguf Q4_K_M
```

Что читать

- Liu et al., 2024 — обзор оптимизаций инференса MoE.
- MoESD и The Limits of Speculation — спекуляция на MoE.
- SpecMoEOff — спекуляция, которая прячет подгрузку экспертов.

5. Nano → Super: каскады и SpecReason

Суть	Группа 3. Часть работы Super отдаём Nano: целые запросы (маршрутизация), отдельные токены (спекулятивные каскады) или шаги рассуждения (SpecReason). В отличие от спекулятивного декодинга, ответ может отличаться от ответа Super.
Перспективность	●●●●○ **4 из 5** — буквально задача «ускорить Super через Nano»; всё мерится через API, но стоит денег и меняет ответы
Что ускоряет в теории	Число токенов и деньги Super. Каскад: если Nano 30B закрывает долю f запросов, расход падает в 1 ÷ (0,27·f + 1 − f) раз, где 0,27 — отношение цен выходных токенов Nano 30B и Super: f = 0,3 → в 1,3 раза, f = 0,5 → в 1,6, f = 0,7 → в 2. SpecReason: при приёме 70–80% шагов и Nano в 3–5 раз быстрее Super — ~1,6–2,4× по задержке в нашей модели; в статье 1,4–3×.
Потери качества	Есть — меряем точность.
Где играть	API Nebius; малой моделью может быть и локальная Nano.
Цена	SpecReason — ~$0,03 на задачу, ~$10 на конфигурацию из 300 задач (контекст ~2 тыс. токенов, 40 шагов). Маршрутизация дешевле.

Варианты

Вариант	Где	Оценка	Комментарий
Маршрутизация запросов: FrugalGPT, RouteLLM	API	●●●●○ 4	Nano отвечает сама, если уверена (по logprobs или по обученному классификатору), иначе — Super.
SpecReason	API	●●●●○ 4	Nano пишет шаг рассуждения, Super ставит ему оценку 0–9 одним токеном — это почти только чтение (~70 новых токенов). При оценке от 7 шаг принят, иначе Super пишет его сам. В статье на моделях 32B — 1,4–3× по задержке и +0,4…9% точности.
Спекулятивные каскады	API, офлайн	●●●○○ 3	Как спекулятивный декодинг, но токен Nano принимается и без совпадения — когда Nano «достаточно хороша» по правилу отсрочки. Проверяется офлайн по logprobs.
Speculative Thinking	API	●●●○○ 3	Наоборот: рассуждает Nano, Super вмешивается в местах сомнений («Wait», «Hmm»).

Почему это важно у нас

- Через API скорость провайдера не наша: по данным Artificial Analysis на Nebius Super выдаёт ~209 ток/с, а Nano 30B — ~168 ток/с. Маршрутизация к Nano там экономит деньги, а не время. Поэтому мерим токены, доллары и «время по модели» — сколько шагов сделала бы каждая модель на нашем железе.
- Ответы Super с logprobs из семейства 1 — общая база: их можно переиспользовать для каскадов и маршрутизации.

С чего начать

- Маршрутизация на 200 задачах: «всё в Super» против «Nano 30B, если средний logprob ответа выше порога, иначе Super» → кривая «точность — доля запросов в Super».
- SpecReason: шаг — абзац рассуждения Nano 30B или Lightning; Super отвечает одним токеном; порог 7.
- Деньги — заранее; Batch API дешевле на 50%, если ответ не нужен сразу.

Ловушки

- Не путать «дешевле» и «быстрее»: через API это разные вещи.
- Варианты и база — на одних и тех же задачах; max_tokens по умолчанию 8192.

6. Разреженность активаций ReLU^2

Суть	После ReLU^2 многие нейроны MLP равны точно нулю, и соответствующие веса down-проекции можно не читать.
Перспективность	●●●○○ **3 из 5** — редкое совпадение с нашей архитектурой и шанс на свой результат; анализ бесплатный, но реальная скорость требует своего ядра
Что ускоряет в теории	Байты на токен. Down-проекция — 21% читаемых весов у 9B и 19% у 4B. При доле нулей s: s = 0,5 → 1,12×; 0,8 → 1,20×; 0,9 → 1,24×, без потерь. Если предсказывать нули заранее и не читать ещё и up-проекцию — до 1,6× при s = 0,9, но уже с потерями.
Потери качества	Нет при точных нулях; есть с предсказателем или порогом.
Где играть	Анализ — на CPU; ядро — llama.cpp (CPU или CUDA) или Triton на T4.
Цена	Анализ — бесплатно; ядро — недели работы разработчика ядра.

В MLP Nemotron нет гейта: $h = \text{ReLU}^2(x \cdot W_{up})$, $y = h \cdot W_{down}$. Если $h_j = 0$, веса W_{down} для нейрона j умножаются на ноль — их можно не читать. У большинства моделей (SwiGLU) точных нулей нет, поэтому TEAL обрезает малые значения по порогу (40–50% разреженности, до 1,5–1,8× на Llama), а Deja Vu учит предсказатель. У нас нули бесплатны: ReLU^2 стоит в MLP у Nano 2 и в экспертах у 30B-A3B.

Почему это важно у нас

- В GGUF веса хранятся блоками по 32 или 256 значений вдоль скрытой размерности MLP. Пропускать можно только целые блоки, поэтому важна доля полностью нулевых блоков, а не отдельных нейронов.
- Помогает перестановка нейронов: «часто нулевые» ставим рядом. Так делает PowerInfer (форк llama.cpp) с «горячими» и «холодными» нейронами.
- ReLU² Wins: ReLU^2 лучше других активаций по разреженности и её предсказуемости.

С чего начать

- 4B на CPU в HF: хуки после активации MLP, доля нулей по слоям на 100 промптах разных задач. Если от 80% — продолжать.
- Доля полностью нулевых блоков по 32 и по 256 подряд — до и после перестановки нейронов по частоте нулей.
- Только потом — ядро, вместе с разработчиком ядра.

Ловушки

- Доля нулей зависит от задачи и длины контекста — мерить на разных задачах.
- Без своего ядра выигрыша не будет: договориться с разработчиком ядра заранее.

7. Движок, сборка и флаги

Суть	Та же модель и те же веса, но другой движок, сборка и настройки.
Перспективность	**обязательный шаг 0** — без этого любое «ускорение» может оказаться ускорением слабого движка. Как исследовательская тема — 2 из 5
Что ускоряет в теории	Накладные расходы. Потолок — таблица в разделе «Откуда берётся время ответа»; разница между «как есть» и потолком — запас этого семейства.
Потери качества	Нет.
Где играть	Везде.
Цена	Часы на настройку; ~0,5 GPU-ч на замеры.

- **llama.cpp** — основной движок на всех устройствах: CUDA на T4, Vulkan на 780M, Metal на Mac, CPU на ПК.
- **HF transformers** — для прототипов и проверки правильности, но не для заявлений о скорости.
- **vLLM на T4** — под вопросом: FlashAttention-2 и FlashInfer требуют sm80+, остаются бэкенды внимания на Triton и Flex — проверить.

Флаг llama.cpp	Что меняет	Где важно
`-ngl N`	сколько слоёв на GPU	T4, 780M
`-fa on|off`	FlashAttention	везде; нужен для квантованного V-кэша
`-sm layer|row`	как делить модель между двумя T4	Kaggle
`-t, -tb`	потоки CPU для генерации и prefill	ПК, Mac, CPU-части
`-ub N`	микробатч для prefill	длинные промпты
`-ctk, -ctv`	тип KV-кэша	только длинные тексты (семейство 10)

С чего начать

- llama-bench для 4B, 9B и 30B-A3B на T4 и на ноутбуке; посчитать долю от потолка.
- 9B на одной T4: llama.cpp против HF transformers — понять масштаб разницы.
- На T4 проверить fp16 на переполнения: NaN, бессмысленный текст, сравнение с ответами на CPU.

```
llama-bench -m nano9b-Q4_K_M.gguf -ngl 99 -fa 0,1 -p 512 -n 128 -r 5
```

Ловушки

- Первый запуск медленнее: прогрев и загрузка в память.
- Ноутбук — от сети и в одном режиме питания; следить за нагревом, частоты плавают.
- На Kaggle не мерить скорость, пока на той же машине идёт другая задача.

8. Пропуск и удаление слоёв

Суть	Выкинуть часть слоёв навсегда (ShortGPT) или пропускать их на «лёгких» токенах (ранний выход).
Перспективность	●●○○○ **2 из 5** — как отдельный метод сильно теряет в качестве; полезен как источник само-черновика (семейство 1) и карта важности слоёв
Что ускоряет в теории	Байты на токен ≈ доля оставшихся слоёв: −25% слоёв → 1,33×; −50% → 2×.
Потери качества	Большие без дообучения.
Где играть	Анализ — CPU и Kaggle.
Цена	Карта важности — дёшево; лечение дообучением — дорого (семейство 12).

- NVIDIA при сжатии 12B → 9B удаляла слои по важности (по MSE) и потом дистиллировала; обрезка голов Mamba сильно била по точности.
- Ранний выход с обучением (LayerSkip) требует переобучения — не для нас.
- Дешёвый полезный результат — карта важности слоёв гибрида по типам (Mamba, внимание, MLP): она пригодится для само-черновика и смешанной квантизации.

С чего начать

- 4B: убирать по одному слою и мерить перплексию и точность на 50 задачах → карта важности.
- Лучших кандидатов на пропуск отдать в само-черновик (семейство 1).
- В llama-quantize есть `--prune-layers`; для гибрида может не сработать — тогда в HF.

9. Параллельная генерация и батчинг

Суть	Генерировать несколько последовательностей за один проход.
Перспективность	●●○○○ **2 из 5** — как тема это про серверы, но как инструмент нужно всем: прогоны качества и параллельные ответы для семейства 2
Что ускоряет в теории	Пропускную способность. При батче 1 T4 упирается в память: ~203 операции на байт против ~3,3, которые нужны одному потоку в Q4. В теории десятки потоков почти бесплатны, на практике рост кончается раньше.
Потери качества	Нет у батчинга; есть у Skeleton-of-Thought — меняется формат ответа.
Где играть	Везде, особенно Kaggle.
Цена	Почти ничего.

С чего начать

- Суммарные ток/с при 1, 2, 4, 8, 16 потоках на T4 и ноутбуке.
- Для семейства 2: 4 коротких ответа без рассуждения против одного длинного — время и точность.

```
llama-batched-bench -m nano9b-Q4_K_M.gguf -ngl 99 -c 16384 -npp 512 -ntg 128 -npl 1,2,4,8,16
llama-server -m nano9b-Q4_K_M.gguf -ngl 99 -np 4   # 4 параллельных слота
```

10. Сжатие KV-кэша и состояния

Суть	Хранить KV-кэш в меньшей разрядности (KIVI), выбрасывать «неважные» токены (SnapKV, H2O) или держать окно и «якорные» токены (StreamingLLM).
Перспективность	●○○○○ **1 из 5** — у нас 4 слоя внимания из 56: KV-кэш мал, а состояние Mamba и так фиксированного размера
Что ускоряет в теории	Трафик KV. KV-кэш 9B — 16 КБ на токен: 8 тыс. токенов — 134 МБ (2,7% трафика Q4-весов, потолок 1,02× при сжатии в 4 раза), 32 тыс. — 537 МБ (1,08×), 128 тыс. — 2,1 ГБ (1,29×). Сжатие состояния Mamba (~142 МБ) в fp16 сэкономит меньше 3% трафика.
Потери качества	Малые при квантовании KV; есть при выбрасывании токенов.
Где играть	Везде.
Цена	Один флаг.

С чего начать: только если появятся тексты от 32 тыс. токенов — `-ctk q8_0 -ctv q8_0` (для квантованного V llama.cpp включает FlashAttention). Обзор семейства — Li et al., 2024.

11. Сжатие промпта и кэш префикса

Суть	Укоротить длинный промпт (LLMLingua-2) или не пересчитывать общий префикс, например системный промпт.
Перспективность	●○○○○ **1 из 5** — промпты у нас короткие, а prefill у Mamba линейный по длине
Что ускоряет в теории	Prefill. Для 9B на T4 prefill — 0,6% времени ответа при промпте 300 и ответе 1000 токенов, 4% — при промпте 2000, 15% — при 8000.
Потери качества	Есть у сжатия промпта; нет у кэша префикса.
Где играть	Везде.
Цена	Почти ничего.

Для гибрида кэш префикса требует снимков состояния Mamba, а не только KV — это тот же откат, что и в семействе 1. Имеет смысл только для задач с длинными документами или RAG.

12. Сжатие с переобучением

Суть	Обрезать модель и вернуть качество дистилляцией (Minitron, Nemotron Elastic), квантовать с обучением (QAT), учить головы-черновики в полном масштабе.
Перспективность	●○○○○ **1 из 5** — сейчас — нет. Мини-версии (LoRA после удаления слоёв, маленькая голова EAGLE-3 для 4B) — 2 из 5 на втором этапе
Что ускоряет в теории	Всё сразу — модель меньше. Но цена — 10^{10}–10^{11} токенов обучения: 4B из 9B — 63 + 150 млрд токенов; Elastic — 110–160 млрд.
Потери качества	Есть, лечатся обучением.
Где играть	Дата-центр; мини-версии — Kaggle.
Цена	Мини-голова EAGLE-3 — по нашей грубой оценке от ~5 GPU-ч (4B, 20–50 млн токенов) до ~40 (9B, от 100 млн).

Отсечено: только для дата-центров

Читать можно — для кругозора. Планировать эксперименты — нет.

Что	Почему не у нас
FP8, NVFP4, MXFP4	Нужны Ada, Hopper или Blackwell; T4 — Turing.
FlashAttention-2/3/4, FlashInfer	Нужен sm80 и новее; к тому же у нас всего 4 слоя внимания.
Ядра Marlin и Machete (в том числе в Quamba2)	sm80+ и sm90.
Ускорение от разреженности 2:4 (SparseGPT, Wanda)	Нужны разреженные тензорные ядра Ampere+; на CPU выигрыша нет.
Тензорный и экспертный параллелизм	Нужны NVLink и много GPU; у двух T4 только PCIe.
Разнесение prefill и decode по машинам, мульти-LoRA, планировщики больших батчей	Это про серверы с тысячами пользователей.
Дистилляция и QAT в полном масштабе	10^{10}–10^{11} токенов обучения (семейство 12).

Наши ресурсы

Ресурс	Что есть	Для чего
Kaggle	На аккаунт ~30 ч в неделю сессий с GPU, до 12 ч за сессию; в сессии две T4 (16 ГБ, ~320 ГБ/с каждая). Квота считается по времени сессии, а не по картам — проверить по счётчику. CPU-сессии GPU-квоту, насколько известно, не тратят	замеры на T4, прогоны качества, мини-обучение
Ноутбук Игоря	Radeon 780M, 32 ГБ общей памяти (~90–120 ГБ/с)	llama.cpp на Vulkan; 30B-A3B в Q4
Мас Руслана, ПК Димы	Metal; CPU и 16 ГБ RAM	малые модели, анализ на CPU, квантизация
Nebius Token Factory	$150. За 1 млн токенов (вход / выход): Super $0,30 / $0,90; Nano 30B и Lightning $0,06 / $0,24. Batch API — на 50% дешевле	Super, качество 30B-A3B и Lightning

В первой сессии Kaggle стоит записать nproc и free -g: сколько CPU и RAM у машины.

Словарик

Термин	Что значит
Prefill и декодирование	Чтение промпта (идёт параллельно, упирается в вычисления) и генерация ответа по токену (упирается в память).
Батч	Сколько последовательностей идут одним проходом. Батч 1 — один пользователь.
ПСП и потолок	Пропускная способность памяти, ГБ/с. Потолок — максимум ток/с = ПСП ÷ байты на токен.
Цель и черновик	Большая модель, которую ускоряем, и дешёвый источник догадок.
γ, α, c, τ	Длина черновика; доля принятых токенов; цена шага черновика относительно шага цели; среднее число токенов за проход цели.
Без потерь	Ответы совпадают с базой (в жадном режиме — побайтно).
Квант, bpw	Формат хранения весов в GGUF (Q8_0, Q4_K_M и др.) и число бит на вес.
Эксперт, top-k	Маленький MLP внутри MoE-слоя; сколько экспертов работает на токен.
Состояние Mamba	Память слоя Mamba фиксированного размера вместо KV-кэша.
logprobs	Логарифмы вероятностей токенов, которые отдаёт API.

Что читать сначала

- LLM Inference Unveiled: Survey and Roofline Model Insights — почему генерация упирается в память и как считать потолки.
- A Survey on Efficient Inference for Large Language Models — общая карта методов.
- Xia et al. — обзор спекулятивного декодинга и Spec-Bench.
- Stop Overthinking — экономия рассуждений.

Источники фактов о моделях

- Megatron-Bridge: конфигурации Nemotron-H и Nano 9B v2 — схема слоёв 9B.
- Карточка Nano 9B v2 — режимы рассуждения, выборка, флаг состояния Mamba.
- Статья Nemotron Nano 2 — ReLU^2, бюджет, сжатие 12B → 9B.
- Nemotron 3 Nano 4B — слои 4B и её дистилляция из 9B.
- Карточка Nemotron 3 Nano 30B-A3B и статья — слои, эксперты, ReLU^2 в экспертах, бюджет.
- Исходники llama.cpp, сборка от 15.10.2026 — флаги и типы спекуляции.